



PluriTAL
Constructions de données 2
2025-2026 (M1)

Projet 7
Classification automatique de
reprises erronées

Enseignante encadrante: Iris Eshkol-Taravella

Etudiantes : Alice LIN, Hongrui LU, Shiyi YAO, Yinbo ZHAO

SOMMAIRE

I. Introduction.....	2
II. Données / Corpus.....	2
2.1 Présentation.....	2
2.2 Prétraitement.....	3
III. Méthodologie.....	4
IV. Expériences & Analyses.....	6
4.1 SVM Linéaire.....	6
4.2 SVM RBF.....	7
4.3 Utilisation d'un dev.....	9
V. Conclusion.....	11
VI. Bibliographies.....	12
VII. Annexes.....	12
7.1 Description du tableau (CSV).....	12

I. Introduction

Projet 7 : Classification automatique de reprises erronées

- Récupérer un fichier csv auprès de Vanessa Gaudray Bouju <v.gaudraybouju@gmail.com>
- Le fichier contient les informations suivantes :
 - reprise
 - son antécédent
 - le contexte antérieur
 - type de reprise
 - d'autres info
- Diviser le corpus en train/test
- Testez un classifieur (réfléchir ou demander l'avis de Vanessa) sur ces données en utilisant les informations comme les features
- L'objectif est de classer les reprises erronées en 3 classes (problèmes grammaticaux, problèmes avec l'antécédent, problèmes avec la reprise)
- Evaluer le classifieur si possible...

Dans ce projet, nous nous intéressons à la classification automatique de reprises erronées. Pour cela, nous utilisons un fichier CSV contenant plusieurs informations sur chaque exemple : la reprise, son antécédent, le contexte antérieur, le type de reprise et d'autres informations.

À partir de ces données, notre travail consiste d'abord à préparer le corpus et à le diviser en ensembles d'entraînement et de test. Nous testons ensuite un classifieur en utilisant les informations fournies comme features.

L'objectif est de classer les reprises erronées en trois classes :

- problèmes grammaticaux,
- problèmes avec l'antécédent,
- problèmes avec la reprise.

Enfin, nous cherchons à évaluer le classifieur, afin de voir dans quelle mesure ces différentes informations permettent de distinguer correctement les trois types d'erreurs.

II. Données / Corpus

2.1 Présentation

Le corpus utilisé dans ce projet a été fourni par Vanessa Gaudray Bouju sous la forme d'un tableau contenant des reprises anaphoriques problématiques annotées dans un corpus de rédactions étudiantes.

Chaque ligne du tableau correspond à un problème de reprise annoté dans le corpus d'origine. Le fichier contient 301 occurrences annotées, réparties dans 133 textes différents. Pour chaque reprise problématique, le corpus fournit plusieurs colonnes décrivant la reprise, son antécédent éventuel, le texte concerné ainsi que différentes caractéristiques linguistiques. De manière générale, ces colonnes

peuvent être regroupées en plusieurs catégories : (Vous trouverez la description précise du corpus en annexes)

- textuelles : `TexteErreur`, `Antecedent`, `Contexte`
- descriptives : `NumeroTexte`, `TypeReprise`, `Antecedent_annotate`
- annotation de l'erreur : `TypeErreur1`, `TypeErreur2`, `SousTypeErreur1`, `SousTypeErreur2`
- quantitatives : `Distance_caracteres`, `Distance_mots`, `Distance_phrases`, `GN_concurrents`, `GN_concurrents_compatibles`
- grammaticales : `Type_pronom`, `Definitude_GN`, `FonctionRep`, `FonctionAnte`
- sémantiques : `Embedding_reprise`, `Embedding_antecedent`, `Similarite_reprise_antecedent`

2.2 Prétraitement

Nous avons choisi la colonne `TypeErreur1` comme variable cible pour la classification, puis supprimé les lignes dont cette valeur était manquante. Ensuite, les attributs ont été séparés en trois groupes : les embeddings, les variables numériques et les variables catégorielles.

Pour les embeddings, nous avons sélectionné `Embedding_reprise` et `Embedding_antecedent`, car ces colonnes fournissent une représentation vectorielle de la reprise et de l'antécédent. Pour les variables numériques, nous avons retenu `Distance_caracteres`, `Distance_mots`, `Distance_phrases` et `Similarite_reprise_antecedent`, puisqu'il s'agit de mesures quantitatives directement exploitables par le modèle. Enfin, pour les variables catégorielles, nous avons utilisé `TypeReprise`, `Type_pronom`, `Fonction_reprise` et `Fonction_antecedent`, car elles décrivent des propriétés linguistiques discrètes nécessitant un encodage spécifique.

Les autres colonnes du corpus n'ont pas été conservées. Certaines variables liées au groupe nominal, comme `Definitude_GN`, `GN_concurrents` et `GN_concurrents_compatibles`, n'avaient pas été retenues dans un premier temps, en raison du nombre important de valeurs manquantes dans `Definitude_GN`. Nous avons néanmoins testé leur ajout au cours des expérimentations, mais les résultats obtenus étaient moins bons. Cela s'explique sans doute par une représentation plus complexe, avec davantage de variables, sans apport d'information suffisamment discriminante. Nous avons donc finalement choisi de ne pas les conserver.

Dans un premier temps, nous avons vérifié le type des données dans chaque colonne à l'aide de dtypes. Les colonnes d'embeddings étaient stockées sous forme de chaînes de caractères dans le fichier CSV, un premier traitement a consisté à convertir ces chaînes en vecteurs NumPy afin de pouvoir les exploiter comme représentations numériques.

Nous avons ensuite rencontré un problème de dimensions hétérogènes : les embeddings n'avaient pas tous la même taille, ce qui empêchait de les regrouper dans une même matrice. Pour résoudre ce problème, nous avons choisi la dimension la plus fréquente pour chaque colonne d'embeddings. Lorsqu'un vecteur était plus long que cette dimension, il était coupé ; lorsqu'il était plus court, il était complété par des zéros.

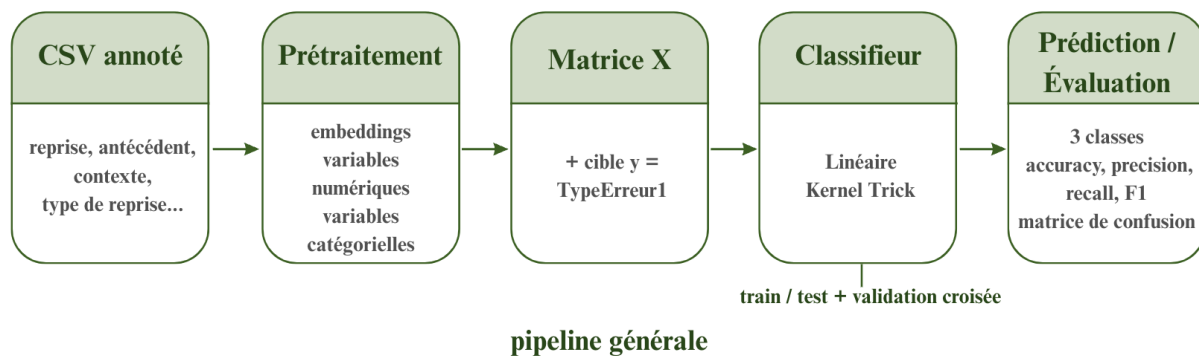
Concernant les variables numériques, nous avons sélectionné les colonnes pertinentes, puis remplacé les valeurs manquantes par 0. Nous avons ensuite appliqué une standardisation par z-score à l'aide de `StandardScaler()`, car les variables numériques peuvent avoir des échelles très différentes d'une colonne

à l'autre. Cette normalisation permet d'éviter qu'une variable ayant des valeurs plus élevées domine la représentation vectorielle finale.

Pour les variables catégorielles, les valeurs manquantes ont d'abord été remplacées par Unknown, puis un encodage one-hot a été appliqué afin de transformer ces catégories textuelles en variables numériques exploitables

Enfin, les trois groupes de variables — embeddings, variables numériques standardisées et variables catégorielles encodées — ont été concaténés horizontalement pour former une unique matrice numérique X, utilisée comme entrée du modèle de classification. La variable cible y correspond aux valeurs de la colonne `TypeErreur1`.

III. Méthodologie

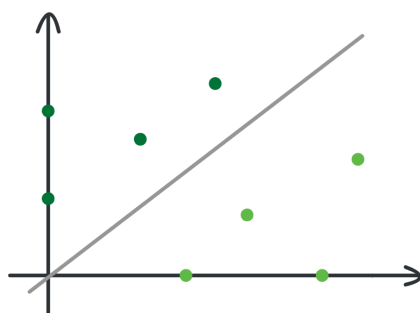


À partir du fichier CSV annoté, nous avons d'abord construit une représentation numérique unique de chaque occurrence en combinant trois types d'informations : les embeddings, les variables numériques et les variables catégorielles. Après prétraitement, ces informations ont été concaténées dans une matrice X, tandis que la variable cible y correspond à la colonne `TypeErreur1`. (Les étapes détaillées du prétraitement ont été présentées dans la section précédente.)

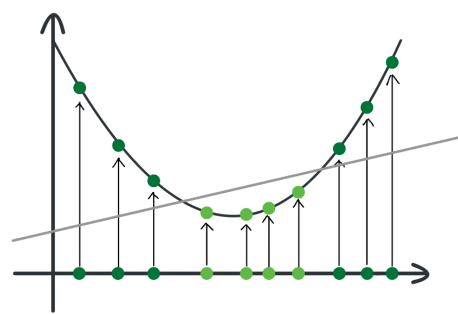
Nous avons ensuite divisé les données en deux parties : 80 % pour l'entraînement et 20 % pour le test. Une validation croisée est appliquée sur les données d'entraînement afin d'obtenir une estimation plus stable des performances.

Pour réaliser la classification, nous avons choisi d'utiliser deux classifieurs étudiés ce semestre pendant le cours *Traitement Statistique des Données* avec Messieurs Dupont et Nouvel : un classifieur linéaire et un classifieur à noyau RBF, fondé sur le kernel trick.

- Le classifieur linéaire repose sur l'idée de séparer les exemples à l'aide d'un hyperplan dans l'espace des attributs.
- Le classifieur à noyau, qui permet de traiter des cas où cette séparation directe n'est pas suffisante, en projetant les données dans un espace de dimension plus élevée.



Classificateur linéaire



Classificateur RBF

Pour l'optimisation, nous avons testé plusieurs paramètres des modèles :

- `C` : pour ajuster le compromis entre apprentissage et généralisation
- `class_weight` : pour mieux prendre en compte un éventuel déséquilibre entre les classes

Enfin, l'évaluation des modèles s'est faite en deux étapes.

D'abord, sur les données d'entraînement, nous avons appliqué une validation croisée à 5 folds pour une estimation plus stable des performances. Concrètement, les données d'entraînement sont divisées en cinq parties : à chaque fois, le modèle est entraîné sur quatre parties et évalué sur la partie restante. L'opération est répétée cinq fois, puis les scores obtenus sont moyennés. Pour cette première évaluation, nous avons surtout regardé l'`accuracy moyenne`, `F1-macro moyen`. Dans certains essais, nous avons aussi pris en compte la `précision macro` et le `rappel macro`.

Dans un deuxième temps, nous avons évalué les résultats obtenus sur les données de test à l'aide d'un `classification report` et d'une matrice de confusion. Le `classification report` présente, pour chaque classe: la `précision`, le `rappel`, le `F1-score`, le `support` (*nombre d'exemples de chaque classe dans test*).

Il donne aussi une `accuracy globale`, ainsi que deux moyennes :

- la `macro avg` : la moyenne des scores calculée en donnant le même poids à chaque classe
- la `weighted avg` : la moyenne des scores pondérée par le nombre d'exemples dans chaque classe

En plus de ce report, nous avons utilisé une matrice de confusion. Cette matrice permet de voir plus concrètement comment le modèle classe les exemples.

Voici un exemple d'évaluation :

Classification report :				
	precision	recall	f1-score	support
E antecedent	0.77	0.74	0.75	27
E grammaticale	0.56	0.60	0.58	15
E reprise	0.72	0.72	0.72	18
accuracy			0.70	60
macro avg	0.68	0.69	0.69	60
weighted avg	0.70	0.70	0.70	60

Confusion matrice :

```
[[20  5  2]
 [ 3  9  3]
 [ 3  2 13]]
```

IV. Expériences & Analyses

4.1 SVM Linéaire

Pour un premier test, nous avons gardé les paramètres par défaut en utilisant que le modèle linéaire simple de SVM. Le score final paraît plutôt faible, avec une moyenne aux alentours de 0,62. Selon les scores obtenus, le meilleur score de classification serait celui de l'antécédent (0,71). Le score pour l'étiquette grammaticale est à près de 0,2 plus faible que l'antécédent (0,53), la reprise se positionne entre les deux avec 0,53.

Après l'étape de prétraitement du corpus, nous avons obtenu une matrice de caractéristiques X de dimension (299, 661), dont le nombre d'exemples est de 299, ce qui indique que le nombre d'exemples est relativement limité par rapport à la dimension de l'espace des caractéristiques. Cette configuration pourrait entraîner un risque de surapprentissage.

Afin d'évaluer l'impact des paramètres du modèle, nous avons réalisé trois expériences en fixant $test_size = 0.2$, $n_splits = 5$ et $max_iter = 50000$, tout en faisant varier les paramètres C et $class_weight$. Nos analyses se concentrent principalement sur les résultats de la validation croisée et du jeu de test (voir le tableau 1).

Exp	Paramètres des expériences	CV F1-macro moyen	Test Accuracy	Test F1-mesure
1	- $test_size = 0.2$ - $n_splits = 5$ - $max_iter = 5000$ - $C=0.05$ - $class_weight = \text{Balanced}$	0.6115	0.70	0.69
2	- $test_size = 0.2$ - $n_splits = 5$ - $max_iter = 50000$ - $C=0.2$ - $class_weight = \text{balanced}$	0.6149	0.7167	0.70
3	- $test_size = 0.2$ - $n_splits = 5$ - $max_iter = 50000$ - $C = 0.2$ - $class_weight = \text{None}$	0.6135	0.7167	0.70

Tableau 1 : Comparaison de résultats SVM linéaire

Expérience 1 ($C = 0,05$, $class_weight = \text{balanced}$)

Les résultats de la validation croisée montrent un F1-macro moyen d'environ 0,61. Cependant, les scores par *fold* présentent une variabilité importante (entre 0,52 et 0,72). Cela témoigne d'une certaine instabilité du modèle, probablement due à la taille réduite du corpus.

Sur le jeu de test, le modèle atteint une exactitude (Accuracy) de 0,70, supérieure à celle observée en validation croisée. Cela suggère une possible variance liée au découpage des données. En termes de

performance par classe, les catégories *E antecédent* ($F1 = 0,75$) et *E reprise* ($F1 = 0,72$) sont mieux reconnues que la catégorie *E grammaticale* ($F1 = 0,58$), qui reste la plus difficile à classer.

Expérience 2 ($C = 0,2$, $class_weight = \text{balanced}$)

Pour améliorer les performances, nous avons augmenté le paramètre C à $0,2$, ce qui a permis d'obtenir une légère amélioration, avec un F1-macro moyen de $0,6149$ en validation croisée. Les variations entre les *folds* restent importantes (de $0,50$ à $0,74$), confirmant une sensibilité du modèle aux partitions des données.

Cependant, sur le jeu de test, l'exactitude atteint $0,7167$. Ce score est meilleur par rapport à l'expérience précédente. Plus précisément, la classe *E antecédent* obtient les meilleures performances ($F1 = 0,77$), tandis que *E grammaticale* reste la plus difficile ($F1 = 0,60$). Ces résultats suggèrent qu'une valeur plus élevée du paramètre C favorise un meilleur ajustement du modèle aux données, tout en atténuant légèrement le sous-apprentissage.

Expérience 3 ($C = 0,2$, $class_weight = \text{None}$)

Afin d'observer l'influence de la pondération des classes, nous avons désactivé le paramètre $class_weight$. Les performances demeurent globalement stables, avec un F1-macro moyen de $0,6135$ en validation croisée. Sur le test l'exactitude est de $0,7167$, soit un résultat identique à celui de l'expérience 2. L'absence de différence notable avec l'expérience précédente indique que le $class_weight$ n'a pas d'impact significatif sur les performances du modèle dans ce corpus.

En conclusion, lorsque nous augmentons la valeur de C , les performances sur le jeu de test s'améliorent légèrement. Cela s'explique par le fait que le modèle parvient à capturer plus précisément les frontières sémantiques au sein des vecteurs de mots (embeddings), augmentant ainsi la précision de la classification. Par ailleurs, la modification des $class_weight$ n'a quasiment aucun impact sur nos résultats. D'ailleurs, parmi ces trois tests, l'expérience 2 présente les meilleurs résultats, tant en validation croisée que sur le jeu de test. Si l'on examine les résultats détaillés de cette expérience, on observe que la catégorie *E antecédent* affiche la performance la plus solide avec un score F1 de $0,77$. Cela démontre que l'*E antecédent* possède des caractéristiques très distinctives dans l'espace vectoriel sémantique, ce qui facilite son identification par le modèle. La catégorie *E reprise* suit avec un score F1 de $0,74$; toutefois, son rappel (Recall) atteint $0,78$, ce qui signifie que le modèle est capable de récupérer la grande majorité des relations de reprise. Enfin, la performance la plus faible concerne la catégorie *E grammaticale* ($F1 = 0,60$). Ce constat est constant sur les trois expériences, confirmant qu'il s'agit de la classe la plus difficile à identifier. L'examen de la matrice de confusion révèle qu'elle est fréquemment confondue avec la *E reprise*, ce qui indique un certain chevauchement (overlap) de leurs frontières sémantiques dans l'espace vectoriel.

4.2 SVM RBF

Dans cette partie, nous intégrons le SVM à noyau RBF afin de comparer un modèle à noyau avec le modèle linéaire. Le paramètre gamma du noyau RBF est laissé à sa valeur par défaut, $scale$, et n'a pas été modifié dans nos expériences afin de mieux contrôler les autres variables.

D'un point de vue général, les résultats du modèle linéaire sont meilleurs que ceux du modèle RBF. Pour le SVM linéaire, l'accuracy en test se situe entre $0,61$ et $0,65$, le F1-macro moyen en validation croisée sur l'ensemble d'entraînement est d'environ $0,62$, et le F1 sur le jeu de test varie

entre 0.60 et 0.64. En revanche, pour le SVM RBF, l'accuracy en test se situe entre 0.43 et 0.58, le F1-macro moyen en validation croisée entre 0.48 et 0.52, et le F1 sur le jeu de test entre 0.41 et 0.55.

Cela peut s'expliquer par le fait que notre corpus est de petite taille (299 lignes), mais le nombre d'attributs est relativement élevé (661). Le noyau RBF repose sur les distances et les similarités entre les exemples, et permet une séparation non linéaire grâce à une projection implicite dans un espace de dimension plus élevée. Cependant, dans un espace déjà très dimensionnel et avec peu d'exemples, cette flexibilité supplémentaire n'apporte pas forcément un gain. Le modèle peut devenir plus sensible au bruit et aux variations locales du jeu d'entraînement, ce qui peut nuire à la généralisation. À l'inverse, le SVM linéaire apprend une frontière de décision plus globale, donc il semble mieux convenir à nos données.

En plus, on contrôle les paramètres `test_size` (exp1 et exp2), `C` (exp1 et exp3), `class_weight` (exp1 et exp4).

En comparant l'exp1 et l'exp2, lorsque `test_size` augmente de 0.2 à 0.3, les résultats des deux modèles baissent légèrement. Toutefois, la diminution est plus nette pour le SVM RBF, dont le F1 en test perd environ 0.06 point. En effet, une augmentation de la taille du jeu de test réduit la quantité de données disponibles pour l'apprentissage. Avec moins d'exemples, le modèle dispose de moins d'informations pour apprendre une frontière de décision fiable. Cet effet semble plus fort pour le noyau RBF, plus dépendant des structures locales des données. Enfin, le F1-macro est plus sensible que l'accuracy, car il évalue séparément la performance sur chaque classe, y compris les classes minoritaires.

En comparant **l'exp1 et l'exp3**, lorsque `C` passe de 1 à 10, les résultats du SVM RBF augmentent légèrement, tandis que ceux du SVM linéaire varient très peu. Cela suggère que, pour le modèle linéaire, l'intervalle de valeurs testé pour `C` reste relativement stable. En revanche, le noyau RBF semble plus sensible à cette variation, ce qui confirme que le modèle linéaire apparaît ici plus robuste aux changements de paramétrage.

En comparant **l'exp1 et l'exp4**, lorsque `class_weight` passe à `balanced`, les performances du SVM RBF sur le jeu de test se dégradent nettement, avec une baisse d'environ 0.12 point. En revanche, les résultats du SVM linéaire changent peu et montrent même une légère amélioration. Cela suggère que l'effet de `class_weight= "balanced"` dépend du modèle considéré. Dans le cas du SVM linéaire, cette pondération semble aider à mieux prendre en compte les classes minoritaires sans modifier fortement la frontière de décision. À l'inverse, pour le SVM RBF, plus sensible aux ajustements locaux, cette pondération peut conduire à une frontière non linéaire moins adaptée au jeu de test.

Nous remarquons toutefois que, pour le SVM RBF, le F1-macro moyen en validation croisée varie très peu. La baisse observée concerne donc surtout le jeu de test. Cela peut indiquer que l'effet de cette pondération ne se manifeste pas de manière nette en moyenne sur les sous-partitions d'apprentissage, mais qu'il dépend davantage de la composition du jeu de test, qui peut être plus difficile, plus bruité ou contenir des classes minoritaires moins bien représentées.

Exp	Paramètres des expériences	SVM RBF			SVM Linear		
		CV F1-macro moyen	Test Accuracy	Test F1-mesure	CV F1-macro moyen	Test Accuracy	Test F1-mesure
1	- test_size = 0.2 - n_splits = 5 - C = 1.0 - class_weight = None	0.4931	0.5667	0.53	0.6221	0.6333	0.62
2	- test_size = 0.3 - n_splits = 5 - C=1.0 - class_weight = None	0.4872	0.5333	0.47	0.6245	0.6111	0.60
3	- test_size = 0.2 - n_splits = 5 - C = 10.0 - class_weight = None	0.5219	0.5833	0.55	0.6266	0.6166	0.61
4	- test_size = 0.2 - n_splits = 5 - C = 1.0 - class_weight = balanced	0.4916	0.43	0.41	0.6262	0.65	0.64

Tableau 2 : Comparaison de résultats SVM RBF et SVM linear

Les matrices de confusion du SVM RBF montrent surtout une confusion entre E antecédent et E reprise. Dans les expériences sans `class_weight=balanced`, plusieurs exemples de E reprise sont prédits comme E antecédent : 9 sur 18 dans l'exp1, 14 sur 28 dans l'exp2 et 7 sur 18 dans l'exp3. La classe E grammaticale reste aussi difficile à reconnaître : seulement 5/15, 4/22 et 5/15 exemples sont correctement classés selon les expériences. Avec `class_weight= "balanced"`, le comportement du RBF change fortement : E antecédent devient la classe la plus mal reconnue, avec seulement 4 exemples corrects sur 27, tandis que 12 sont prédits comme E reprise et 11 comme E grammaticale. Ces résultats montrent que le SVM RBF est sensible au paramétrage et que ses erreurs se déplacent fortement selon les réglages, alors que le SVM linéaire reste plus régulier.

4.3 Utilisation d'un dev

	Dev	Test Final	Test Final (balanced)
E antecédent	0.67	0.77	0.70
E grammaticale	0.69	0.56	0.50

	Dev	Test Final	Test Final (balanced)
E reprise	0.60	0.72	0.62

Tableau 3 : Entraînement dev sur la base de LinearSVC

En reprenant le script [lineaire.py](#), nous avons ajouté un entraînement sur le jeu de développement (dev) afin que le modèle puisse optimiser ses paramètres avant de les appliquer sur le jeu de test. Par défaut, la division train/dev/test est de 8/1/1. Pour ce premier test, le modèle a sélectionné une faible régularisation ($C=0.1$). En comparant les résultats obtenus sur le jeu de dev et sur le test final, on constate que le modèle a effectivement bénéficié de cet entraînement supplémentaire.

Le score moyen de classification est de 0,65. Par rapport au tout premier script, ce résultat est légèrement inférieur. Parmi les trois catégories, c'est la grammaticalité qui régresse le plus, tandis que les deux autres affichent une légère amélioration après l'entraînement sur le dev.

Nous avons ensuite cherché à améliorer les scores en introduisant un rééquilibrage automatique des poids de chaque classe sur l'ensemble du jeu de données, via les fonctions SVM de scikit-learn. À notre surprise, cette modification n'a pas permis d'améliorer les résultats. Les scores sur le dev restent identiques, mais sur le test final, ils sont inférieurs à ceux obtenus sans rééquilibrage. En classant les résultats par ordre croissant : dev < test final (équilibré) < test final. La valeur de [E_grammaticale](#) demeure systématiquement inférieure à celle obtenue sur le dev, ce qui indique que le modèle peine particulièrement à apprendre pour cette classe.

		Expérience 1		Expérience 2		Expérience 3	
		Dev	Final	Dev	Final	Dev	Final
E antecédent	Linéaire	0.60	0.69	0.69	0.71	0.60	0.69
	RBF	0.61	0.42	0.59	0.61	0.62	0.58
E grammaticale	Linéaire	0.58	0.62	0.67	0.50	0.58	0.62
	RBF	0.86	0.42	0.86	0.38	0.86	0.48
E reprise	Linéaire	0.50	0.61	0.63	0.62	0.50	0.61
	RBF	0.67	0.62	0.62	0.62	0.62	0.53

Tableau 4 : Ajout de dev pour le modèle Linear et RBF

Par la suite, nous avons fusionné les deux scripts (linéaire et RBF) en y intégrant un entraînement sur le dev pour un réglage automatique des paramètres. Par rapport aux résultats précédents, on observe

que les scores finaux sont plus homogènes pour `LinearSVC`. Par ailleurs, la différence entre le modèle linéaire et le modèle RBF reste relativement faible, à l'exception de la classe grammaticale.

Un point notable : lors de l'entraînement sur le dev, le score pour la classe grammaticale est particulièrement élevé, atteignant 0,86 avec le modèle RBF. Cela suggère que le modèle apprend efficacement sur ce jeu de données, et que la classification grammaticale bénéficie davantage du noyau RBF. En revanche, lorsque les paramètres entraînés sur le dev sont appliqués au test, les performances chutent significativement — le score tombe à 0,42 pour la classe grammaticale du modèle RBF. Ce phénomène s'explique par la différence de distribution entre les jeux dev et test : les paramètres optimisés pour l'un ne se généralisent pas nécessairement à l'autre. C'est d'ailleurs pour cette raison que le script `lineaire.py` original obtenait de meilleurs scores sur le test, ayant été directement adapté à ces données.

L'activation du rééquilibrage n'apporte pas d'amélioration significative ; les scores diminuent également, même si l'ampleur de cette baisse varie selon les modèles et les classes. L'analyse des matrices de confusion confirme que le modèle linéaire est globalement plus stable que le RBF. Dans plusieurs configurations, le RBF tend à favoriser la classe *antécédent*, mais ce comportement change fortement avec le rééquilibrage. Le problème persistant sur la classe grammaticale s'explique vraisemblablement par un déséquilibre entre les classes dans les données, que le paramètre `class_weight` ne parvient pas à corriger entièrement.

Pour tenter de résoudre ce compromis, nous avons élargi la grille de valeurs testées pour le paramètre `C`, permettant ainsi au modèle de choisir entre une approche plus généraliste ou une régularisation plus faible favorisant un meilleur ajustement aux données. Si le problème n'est pas entièrement résolu, quelques améliorations sont observées : le modèle parvient partiellement à rattraper ses erreurs sur les classes grammaticale et reprise, et adopte une posture légèrement plus généraliste. Toutefois, le gain en accuracy globale reste limité, ce qui ne permet pas de conclure à un apprentissage pleinement satisfaisant.

Il convient également de mentionner que, lors de l'entraînement sur le dev, nous avons laissé la liberté au modèle RBF de choisir une valeur de gamma autre que `"scale"`. Cependant, pour l'ensemble des tests effectués, son choix final s'est systématiquement porté sur `"scale"`, suggérant que cette valeur par défaut reste la mieux adaptée aux données utilisées.

V. Conclusion

Après plusieurs cycles d'entraînement et de réglage des paramètres, nous pouvons tirer plusieurs conclusions de cette expérience.

La principale limitation rencontrée est le faible volume du jeu de données, qui ne compte qu'environ 300 lignes. Ce manque de données a eu des répercussions à plusieurs niveaux : la répartition déséquilibrée entre les classes biaise le modèle vers la classe *antécédent*, mieux représentée, et la division en sous-corpus train/dev/test produit des ensembles trop hétérogènes pour permettre une bonne généralisation des paramètres appris.

Malgré ces contraintes, la comparaison des deux modèles SVM a permis de dégager des tendances claires. `LinearSVC` s'est révélé mieux adapté à notre jeu de données, offrant une classification plus cohérente et des scores plus stables entre les classes. Le modèle RBF, bien que théoriquement

puissant, nécessite un volume d'exemples suffisant pour tirer parti de ses calculs de distances et de similarités — condition que notre corpus ne remplit pas. Il reste néanmoins très performant sur la classe *antécédent*, ce qui illustre bien sa sensibilité à la distribution des données.

Ces résultats soulignent l'importance de la qualité et de la quantité des données dans une tâche de classification. Pour améliorer les performances, plusieurs pistes seraient envisageables : enrichir le corpus avec davantage d'exemples annotés, notamment pour les classes *grammaticale* et *reprise*, ou encore explorer des approches de rééchantillonnage plus avancées pour pallier le déséquilibre entre les classes. L'utilisation de modèles plus robustes aux petits corpus, comme des classifieurs bayésiens ou des modèles pré-entraînés de type transformeur, pourrait également constituer une direction intéressante pour de futurs travaux.

VI. Bibliographies

- **LinearSVC**
<https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>
- **Traitement statistique des données_2021-04-29 — SVM, modèles d'étiquetage**
<https://loicgrobol.github.io/intro-fouille-textes/>
- **GitHub (dépôt du devoir)**
https://github.com/Shiyi-YAO/Construction_projet

VII. Annexes

7.1 Description du tableau (CSV)

Colonne	Description
TexteErreur	le segment annoté constituant une reprise problématique
TexteAnte	le ou, dans de rares cas, les segment(s) antécédent(s), lorsqu'ils sont annotés
NumeroTexte	l'identifiant du texte concerné
Offsets_Reprise	position de la reprise dans le texte ; cette colonne n'est pas utile dans notre travail
Offsets_Antecedent	position de l'antécédent dans le texte ; cette colonne n'est pas utile dans notre travail

Colonne	Description
TypeReprise	le type de reprise
TypeErreur1	la catégorie d'erreur principale ; c'est la variable cible à prédire
TypeErreur2	renseigné dans les quelques cas où une erreur reçoit deux labels
SousTypeErreur1	la sous-catégorie d'erreurs ; elle peut éventuellement faire l'objet d'une autre prédiction, mais elle ne doit pas être utilisée comme feature pour prédire le type principal
SousTypeErreur2	renseigné dans les cas où une erreur reçoit deux labels
Antecedent_annotate	indique si l'antécédent a été annoté (oui / non / incertain)
DistanceCarac	distance, en nombre de caractères, entre la reprise et l'antécédent, lorsque celui-ci est annoté
DistanceMot	distance, en nombre de mots, entre la reprise et l'antécédent, lorsque celui-ci est annoté
Distance_phrases	distance, en nombre de phrases, entre la reprise et l'antécédent, lorsque celui-ci est annoté
GN_concurrents	nombre de groupes nominaux, autres que la reprise et son antécédent, présents entre le début de la phrase de l'antécédent et la reprise
GN_concurrents_compatibles	nombre de groupes nominaux, autres que la reprise et son antécédent, présents entre le début de la phrase de l'antécédent et la reprise et compatibles avec celle-ci en genre et en nombre
Type_pronom	si la reprise est un pronom, cette colonne précise le type de pronom (défini / démonstratif / relatif)
Definitude_GN	la définitude des groupes nominales
FonctionRep	fonction grammaticale de la reprise, annotée avec Spacy
FonctionAnte	fonction grammaticale de l'antécédent, lorsque celui-ci est annoté
Embedding_reprise	embedding de la reprise
Embedding_antecedent	embedding de l'antécédent
Similarite_reprise_antecedent	similarité cosinus entre la reprise et l'antécédent
Contexte	texte brut du document concerné, sans balises